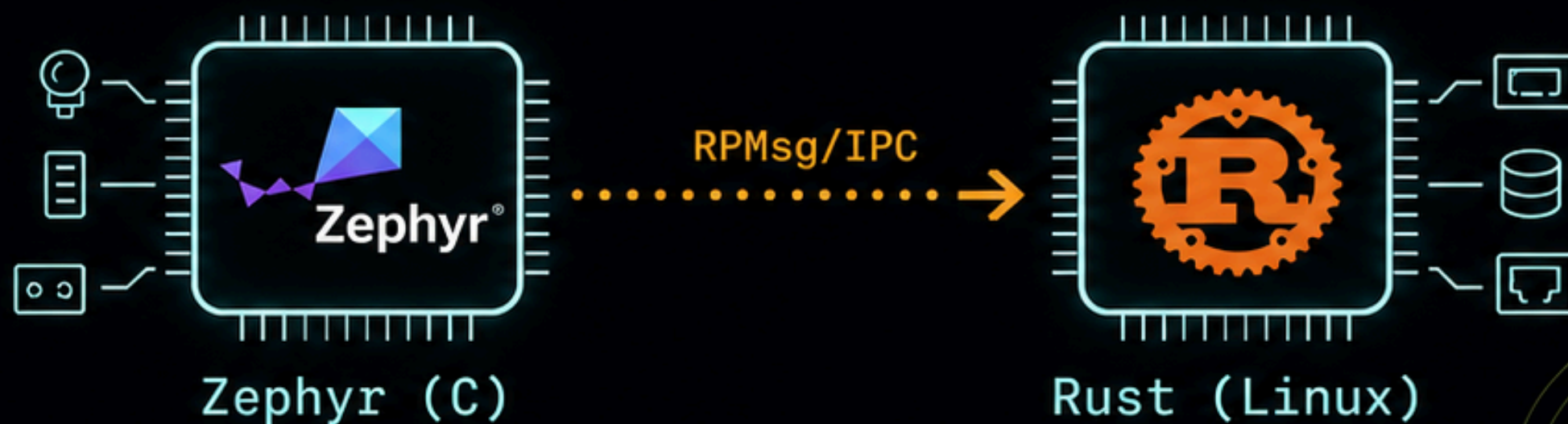


"Hello, IPC"

Connecting Rust and Zephyr



Chrispine Tinaga
Nairobi Linux User Group
September 2026



Hello, Crispine

- ▶ Embedded Systems Engineer
- ▶ Zephyr RTOS, Rust, Python, and EMS
- ▶ Open-source contributor
- ▶ Today: a live embedded demo, plus two real pull requests



AT A GLANCE

- ✓ **zephyrproject-rtos/zephyr** #118106
watchdog driver Cortex-M fix
- **pymodbus-dev/pymodbus** #3000
socket reconnect fix
- ✓ **zephyrproject-rtos/zephyr** #113302
BL350 board support
- ✓ **zephyrproject-rtos/zephyr-lang-rust** #132
Rust blinky sample fix
- **PyPSA/PyPSA** #1151
pandas dtype fix

We'll cover

- 01 Why chips have more than one brain
- 02 Zephyr and Rust, in one sentence each
- 03 The idea: a coprocessor is just a file
- 04 Live demo on real hardware
- 05 What broke, and what it taught me
- 06 Two pull requests that got this here



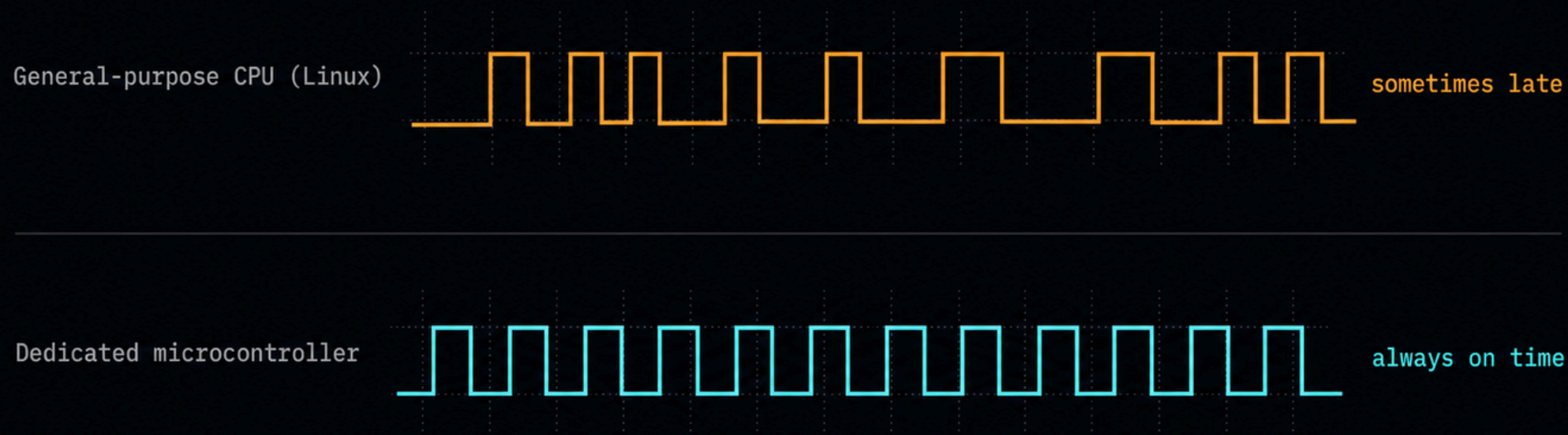
THE PROBLEM

Linux is not good at promises about time

- ▶ Linux schedules your process when it gets around to it
- ▶ “Usually within a millisecond” $\neq$ “always”
- ▶ Some jobs need guarantees: hold a pulse for 2 ms, sample every 500 μ s

TAKEAWAY

Pair a general-purpose core with a small microcontroller that runs nothing else.



Two runtimes, one chip



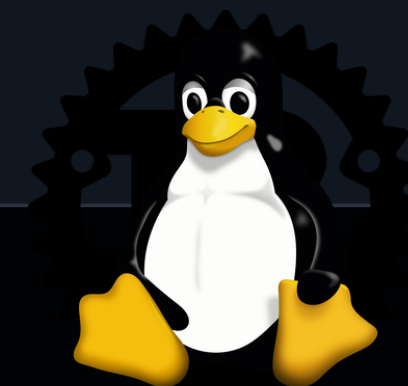
ZEPHYR

- Small open-source RTOS
- Runs on the microcontroller core
- Predictable, real-time-oriented
- Core written in C, with growing Rust support



RUST (on Linux)

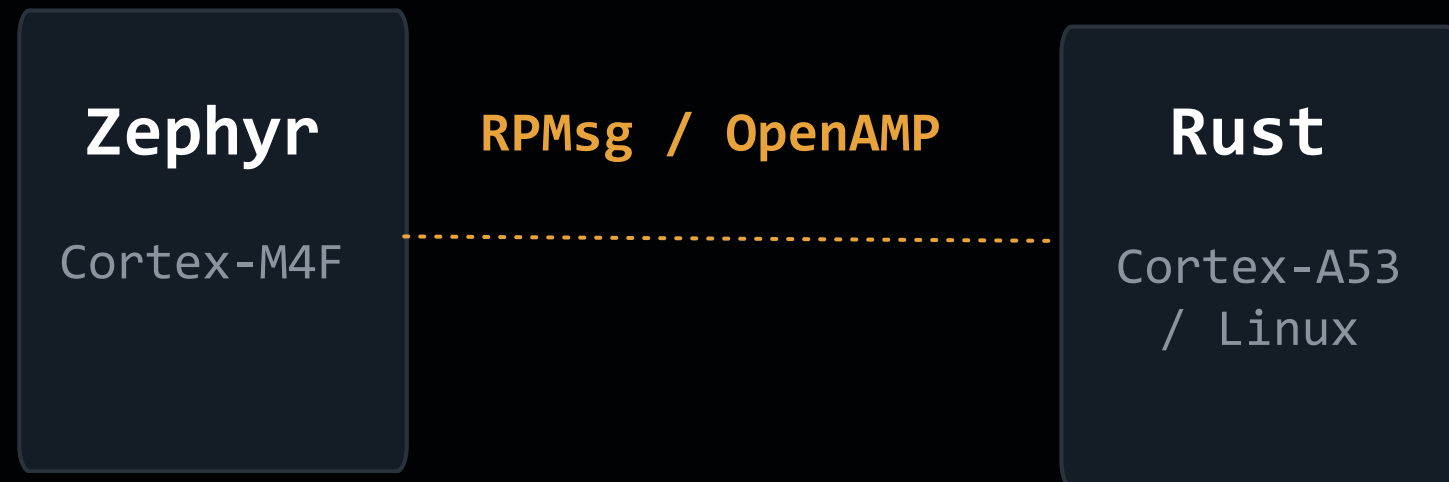
- Memory-safe systems language
- Runs on the Linux application core
- No garbage collector, fast, reliable
- Talks to the world like a normal app



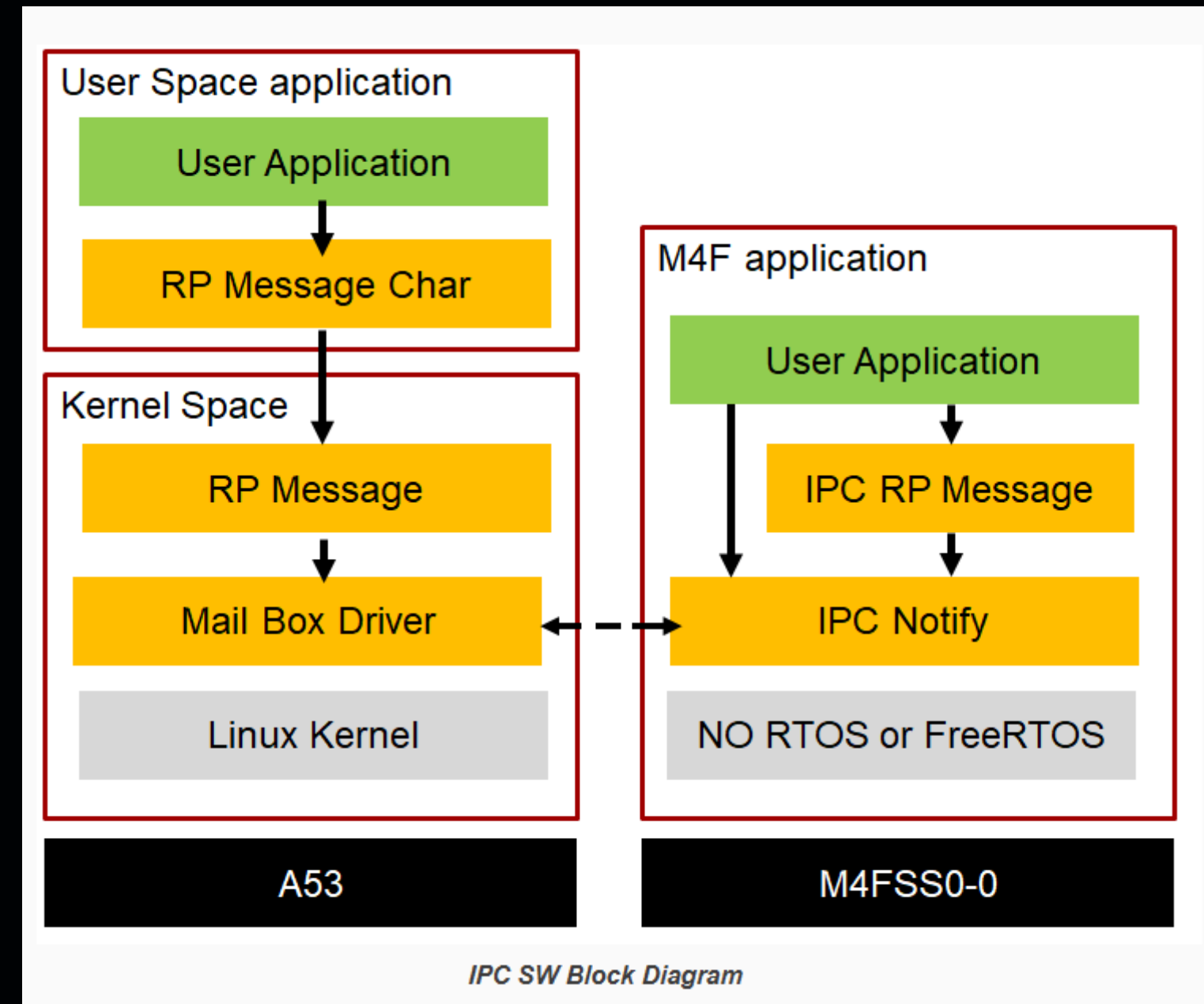
THE CONCEPT

What is IPC, really?

Inter-Process Communication: two programs, exchanging messages



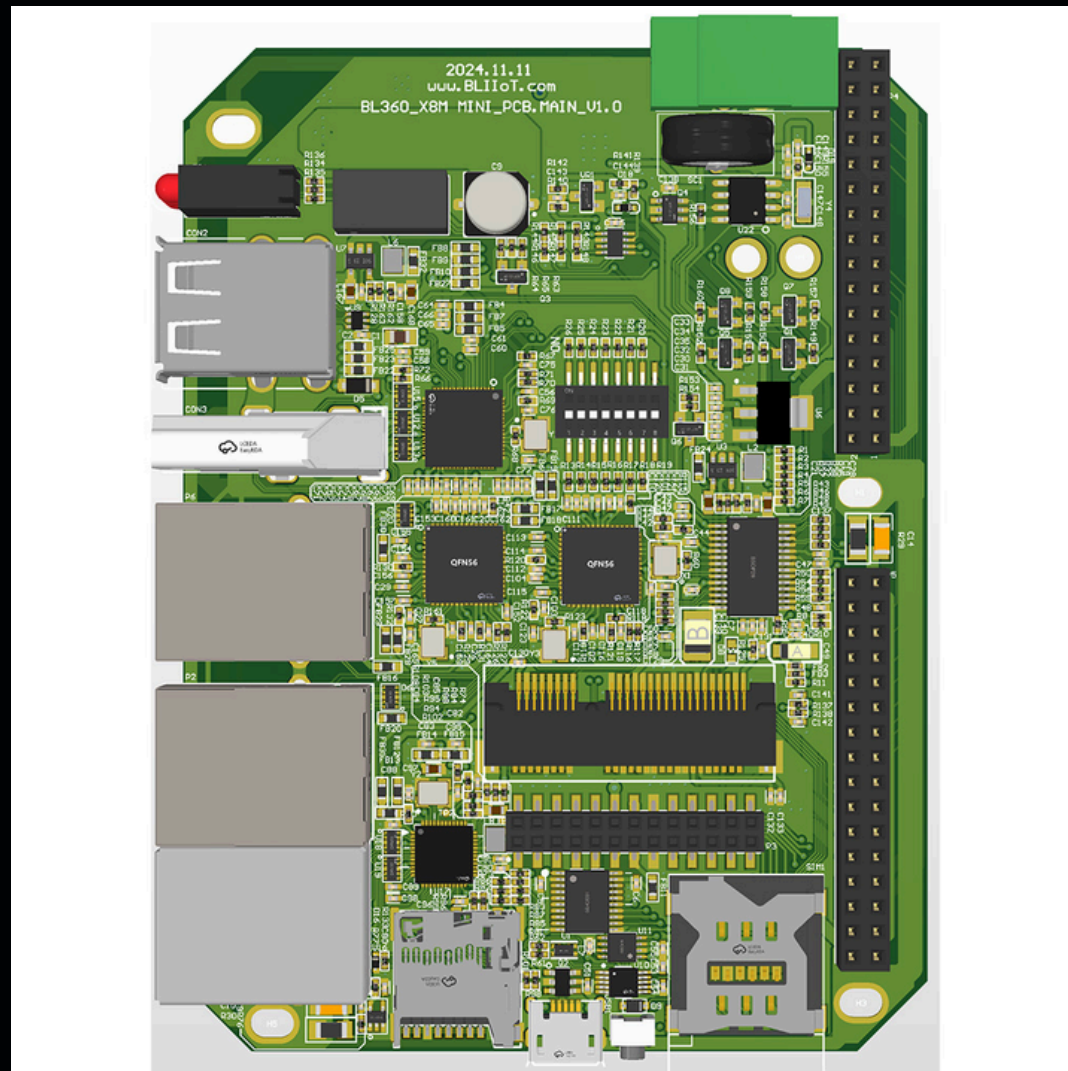
Same chip. Shared memory. Two completely different worlds, one conversation.



Source: software-dl.ti.com

The board

- Industrial embedded controller



BOARD SPEC

am62x_m4_b1350

SOC	TI AM6254
LINUX CORES	4x Cortex-A53 @ 1.4 GHz
RTOS CORE	1x Cortex-M4F @ 400 MHz
SHARED IPC REGION	1 MB (OpenAMP / RPMsg vrings)
STATUS	Maintained, mainline Zephyr

Source: [Zephyr documentation](#)

- I upstreamed this board into mainline Zephyr



On Linux, the coprocessor is a file

```
let mut endpoint = OpenOptions::new()
    .read(true).write(true)
    .open(path)?;

endpoint.write_all(b"Hello from Rust"?);
let n = endpoint.read(&mut buf)?;
```

No driver. No ioctl. No crate.

std + a character device. That's the whole program.

Let's watch it talk

- Rust on Linux (A53) sends a greeting over RPMsg
- Zephyr on the coprocessor (M4F) replies with its uptime
- Verified live on the real am62x_m4_bl350 board

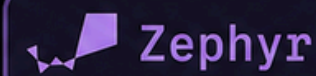
tinegachris / bl350-zephyr-rust-demo

Zephyr ↔ Rust IPC demo on the BLIIoT BL350 (AM62x)

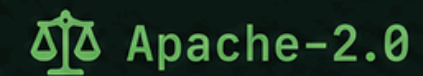
	a53/	Rust (A53 / Linux)
	m4f/	Zephyr (M4F / RTOS)
	docs/	Slides + documentation
	scripts/	
	udev/	
	west.yml	
	README.md	



Rust



Zephyr



Apache-2.0

It started with moving a few lines

- ▶ Upstream PR in the Rust + Zephyr ecosystem
- ▶ samples: rust: blinky: Use conditional imports to fix compiler warnings
- ▶ Literally just moved imports and fixed a warning
- ▶ Proof that your first contribution doesn't have to be huge

RUSTLANG FIRST PR

[zephyrproject-rtos/zephyr#132](#)
Use conditional imports to fix compiler warnings

```
samples/blinky/src/lib.rs
10 10
11 11 use log::warn;
12 12
13 - use zephyr::raw::ZR_GPIO_OUTPUT_ACTIVE;
14 - use zephyr::time::{sleep, Duration};
15 -
16 13 #[no_mangle]
17 14 extern "C" fn rust_main() {
18 15     unsafe {
19 16         @@ -26,6 +23,9 @@ extern "C" fn rust_main() {
20 17
21 23     #[cfg(dt = "aliases::led0")]
22 24     fn do_blink() {
23 25         + use zephyr::raw::ZR_GPIO_OUTPUT_ACTIVE;
24 26         + use zephyr::time::{sleep, Duration};
25 27         +
26 28         warn!("Inside of blinky");
27 29
28 30         let mut led0 = zephyr::devicetree::aliases::led0::get_instance().unwrap();
29 31
30 31
```



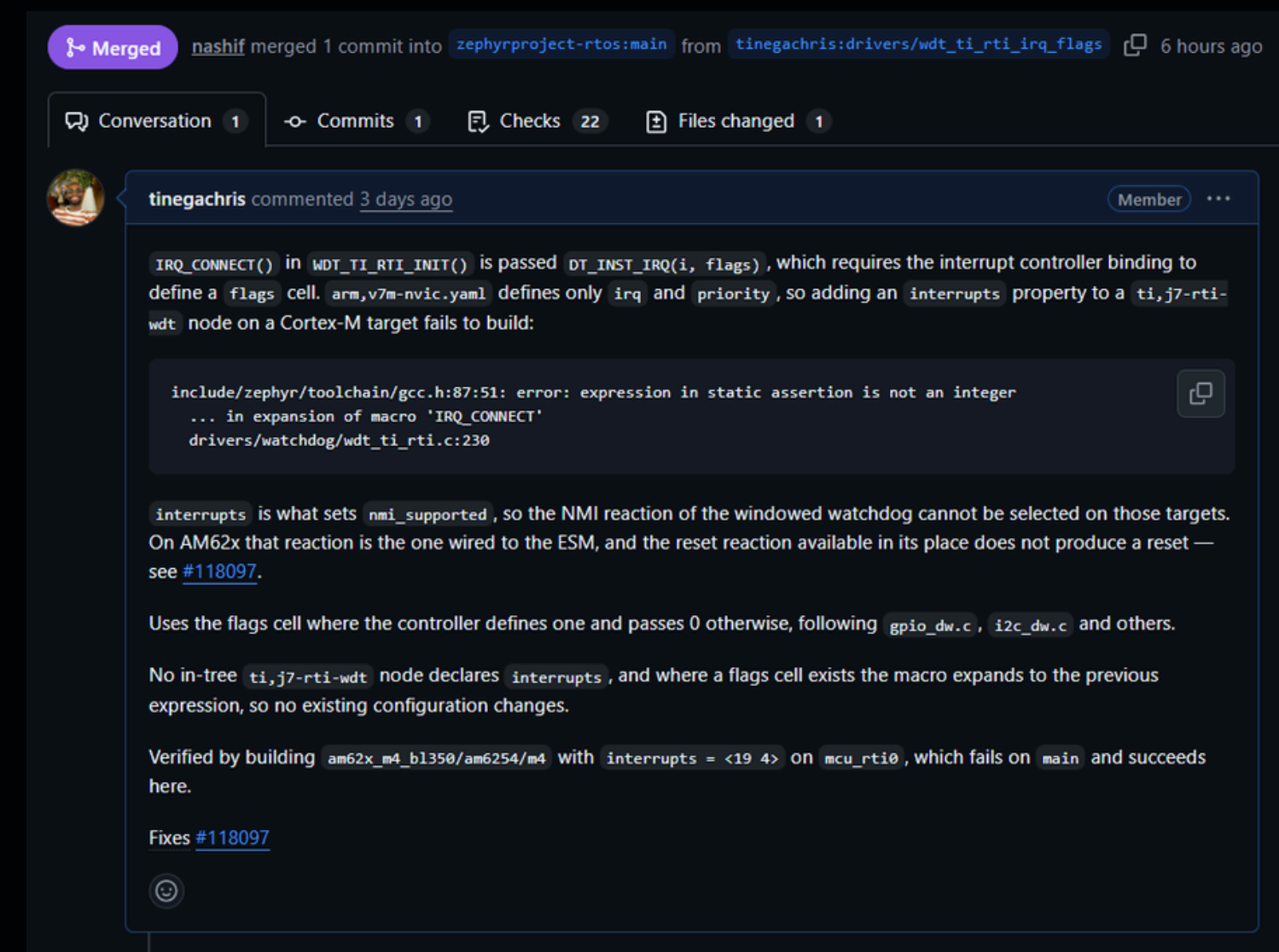
Tiny cleanups still matter,
and they teach you the review process.

Now it fixes real hardware

- ▶ Recent merged PR in Zephyr upstream
- ▶ drivers: watchdog: wdt_ti_rti: fix build with interrupts on Cortex-M
- ▶ Makes the AM62x windowed watchdog build correctly with interrupts
- ▶ Same contributor now fixing real bugs

MOST RECENT MERGE

zephyrproject-rtos/zephyr#118106 Watchdog fix on Cortex-M



Start where I started

One typo fix, one missing doc, one small driver bug.

